

LAB 5: Inheritance in C++

The objective of Lab 5 was to implement linear regression in C++ by reimplementing the supervised-learning framework from previous labs in C++, following the design seen in seminar 4. The goal was to translate the Java implementation into an object-oriented C++ version while keeping the same architecture. The classes `Vector`, `Record`, `Dataset` and `Model` were already implemented and are provided in the supplied code. We implemented the remaining four classes (`Algorithm`, `GradientDescent`, `StochasticGradientDescent` and `SupervisedLearner`) and updated the main function in `TestLearner.cpp` to test all classes.

The final program can compute gradients, apply different optimization algorithms, learn a linear regression model, print parameters, and evaluate accuracy on new inputs.

1. Classes

- A `Dataset` class that stores a collection of records, each containing input features and an expected output.
- A `Record` class that represents a single data sample.
- A `Vector` class used to handle numerical vectors and basic operations required for learning.
- An abstract `Algorithm` class that defines the interface for optimization algorithms.
- Two concrete optimization algorithms: `GradientDescent` and `StochasticGradientDescent`.
- A `Model` class that represents a linear predictive model.
- An abstract `SupervisedLearner` class that connects a model with a learning algorithm.
- A test program (`TestLearner.cpp`) that validates the behavior of the implemented classes.

The final program is able to train a linear model using either batch gradient descent or stochastic gradient descent and evaluate the learned parameters on a dataset.

2. Design Choices and Alternative Solutions

Alternative Solutions Considered

One possible approach to solving this problem would have been to implement all the learning logic inside a single class. For instance, the model, dataset handling, and optimization algorithm could have been tightly coupled in one implementation. While this approach might reduce the number of classes, it would result in poor modularity, limited extensibility, and code that is difficult to maintain or reuse.

Another alternative would have been to implement separate learning algorithms without a common interface. This would make it harder to switch between different optimization strategies or to add new ones in the future, since the rest of the code would need to be modified accordingly.

Chosen Solution

The chosen solution is based on an object-oriented design that emphasizes abstraction, inheritance, and polymorphism. The learning process is decomposed into independent components, each with a well-defined responsibility.

The Algorithm abstract class defines a common interface for optimization methods. Both GradientDescent and StochasticGradientDescent inherit from this class and implement their own version of the optimization procedure. This design allows the learning algorithm to be easily replaced without modifying the rest of the system.

Similarly, the SupervisedLearner class acts as a high-level abstraction that combines a Model with an Algorithm. This separation allows the same learning framework to be reused with different models or optimization strategies. The Model class encapsulates the parameters of a linear model and provides methods for prediction and parameter updates.

The Dataset, Record, and Vector classes are designed to encapsulate data representation and mathematical operations. This avoids spreading low-level data manipulation logic throughout the code and improves readability.

Object-Oriented Concepts Applied

Several object-oriented programming concepts were applied in this lab:

- **Abstraction:** Abstract base classes (Algorithm, SupervisedLearner) define interfaces without committing to specific implementations.
- **Inheritance:** Concrete algorithms inherit from the Algorithm base class.
- **Polymorphism:** Different optimization algorithms can be used interchangeably through base class pointers.
- **Encapsulation:** Internal details of models, vectors, and datasets are hidden behind clear public interfaces.
- **Extensibility:** New optimization algorithms or models can be added with minimal changes to existing code.

This design was chosen because it results in clean, modular, and easily extensible code, which aligns well with the principles taught in the course.

3. Conclusion

The implementation was tested using the provided TestLearner.cpp file, which verifies the correct interaction between datasets, models, and learning algorithms. The tests confirm that both Gradient Descent and Stochastic Gradient Descent are able to update the model parameters correctly and reduce the prediction error over time.

In practice, the solution worked as expected. Gradient Descent produced stable and consistent updates, while Stochastic Gradient Descent converged faster but showed more variability due to its randomized nature. This behavior is consistent with the theoretical properties of these algorithms.

One of the main difficulties during the implementation was correctly managing the interactions between the different classes while keeping responsibilities clearly separated. Designing clean interfaces for the abstract classes required careful thought, especially to ensure that derived classes had all the information they needed without breaking encapsulation.

Another challenge was implementing vector operations in a safe and reusable way, as small mistakes in numerical computations can easily lead to incorrect learning behavior.

Overall, the lab provided a good opportunity to apply object-oriented programming concepts to a machine learning context. The final design is flexible, easy to extend, and demonstrates a clear separation of concerns. The tests indicate that the solution is correct and meets the requirements of the assignment.